PRIVAGENT

SIH 2026 — Winning Implementation Blueprint

On-device Visual Perception for Light-weight Browser Agents

Mission: Let an AI browser agent understand and operate websites while keeping sensitive user information on the user's device. The project must prove privacy, preserve task success, and verify that raw sensitive values do not leave the device.

What the project is	What it is NOT
A local privacy firewall for AI browser agents	Not merely a PII detector
DOM + visual perception + semantic sanitization	Not simple screenshot black-boxing
Local reversible aliases + agent execution	Not sending raw PII to a cloud LLM
Measured privacy–utility trade-off	Not an unverified “privacy” claim

Important positioning: The supplied SIH dossier explicitly warns that WebPII/WebRedact already addresses fast visual PII localization. Therefore the differentiator is the complete privacy-preserving agent pipeline: local perception, semantic sanitization, local identity mapping, network-level leakage testing, and successful browser-task execution after sanitization.

1. Problem Understanding

The official problem asks for a browser-based privacy-preserving vision/agent system. A lightweight local model evaluates the current screen and sensitive visual/PII information before any relevant context is sent to a server-side agent. The target environment includes browser pages where information can appear in normal DOM elements, rendered graphics, images, canvas, or visually complex layouts.

Core pipeline

```
Browser page → local DOM/visual perception → sensitivity analysis → privacy transformation → sanitized agent context → remote/local agent reasoning → structured browser action → local resolution of protected values → browser action.
```

Winning problem statement in one sentence

“How can an AI browser agent complete useful tasks with high task success while the real sensitive values required by those tasks remain local and demonstrably absent from outgoing agent context?”

Design principles

- Privacy by default: raw sensitive values never need to be sent to the agent.
- Minimum necessary disclosure: expose only the semantic information needed for the current task.
- Local-first perception: sensitive data detection should happen on-device whenever feasible.
- Utility preservation: sanitization must not make the browser agent useless.
- Verifiability: measure and demonstrate leakage instead of merely claiming privacy.
- Graceful degradation: support CPU inference when WebGPU is unavailable.
- Use synthetic/fake secrets in demonstrations and benchmarks; never use real personal data.

2. Product Definition — PrivAgent

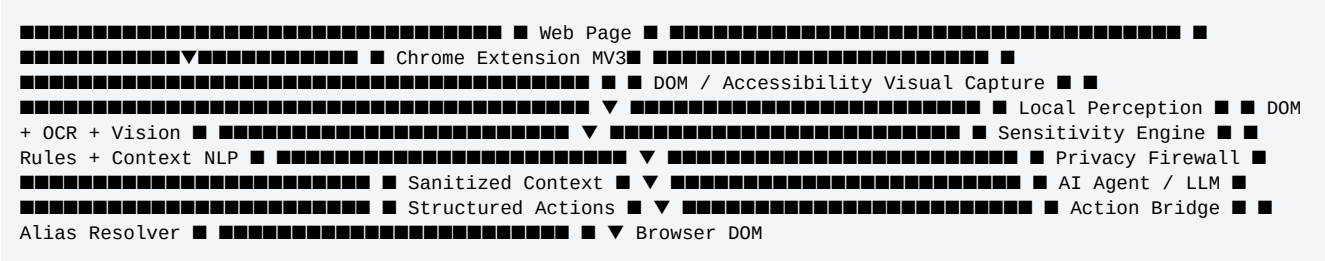
User experience

- The user installs a Chromium/Chrome extension.
- The extension observes the active page using DOM/accessibility information and selected visual captures.
- Local OCR/vision/NLP components identify sensitive regions and estimate sensitivity.
- Sensitive values are replaced by stable semantic aliases such as USER_EMAIL_1.
- The remote AI agent receives sanitized context and decides what browser action to perform.
- When an action needs a protected value, the extension resolves the alias locally and injects the real value into the target element.
- A privacy dashboard shows what was detected, what was protected, what was sent, and measured leakage/task success.

The six features to make the project distinctive

Feature	Why it matters	Required proof
1. Local multimodal perception	Combines DOM/visual/OCR signals rather than relying on one source.	Detects PII in normal DOM and visual-only cases.
2. Semantic sanitization	Preserves meaning for the agent instead of destroying context with blanks.	Agent still understands USER_EMAIL_1.
3. Local identity vault	Keeps alias → real-value mapping on-device.	Remote logs contain no mapping.
4. Privacy firewall	Inspects outgoing agent context before transmission.	Raw secret is blocked/replaced.
5. Leakage sentinel	Turns privacy into a measurable security claim.	Known fake secrets are tested against outgoing payloads.
6. Privacy–utility optimizer	Shows the trade-off between protection and task completion.	Compares full redaction vs semantic sanitization.

3. Reference Architecture



Module responsibilities

Module	Input	Output
Page Collector	DOM, labels, input types, accessibility info	Structured page representation
Visual Collector	Selected viewport/screen captures	Image regions
OCR	Image regions	Text + bounding boxes + confidence
PII/Sensitivity Engine	Text + DOM + context + coordinates	Sensitive entities + confidence + reasons
Sanitizer	Sensitive entity list + page context	Redacted/aliased representation
Local Vault	Alias/value pairs	Local-only mapping
Agent Gateway	Sanitized context + task	Agent response / action plan
Action Bridge	Structured actions + aliases	Browser actions
Leakage Sentinel	Known fake secrets + outgoing payloads	Leakage verdict + metrics
Dashboard	Audit events + metrics	Human-readable privacy/utility UI

4. Recommended Tech Stack

Layer	Recommended	Purpose
Browser	Chrome / Chromium	Primary SIH demo target
Extension	Manifest V3	Modern extension architecture
Extension language	TypeScript	Typed, maintainable browser code
UI	React + Tailwind CSS	Side panel/dashboard
Build	Vite	Fast development/build pipeline
Browser state	Zustand or simple service state	Small predictable state layer
DOM layer	Content scripts + Chrome scripting APIs	Page inspection and action bridge
Visual capture	Chrome tab/screen capture APIs	Visual-only content
OCR	PaddleOCR or browser-compatible OCR	Text localization from images
Local inference	ONNX Runtime Web	Run models in browser
Acceleration	WebGPU with CPU fallback	Lightweight local inference
NLP	Rules + small transformer if needed	Contextual sensitivity classification
Local storage	IndexedDB / extension storage	Aliases, settings, audit metadata
Backend	Python + FastAPI	Agent orchestration and benchmark services
Agent	LLM + constrained structured tools	Reasoning and browser task planning
Model training	Python ecosystem	Offline training/evaluation only
Benchmark	Synthetic webpages + task runner	Repeatable privacy/utility experiments
Testing	Playwright + unit/integration tests	Browser and agent evaluation

Important implementation rule

The backend must not be the place where raw PII is cleaned. Raw PII should be detected and transformed before the remote agent receives context. Python/FastAPI is for orchestration, evaluation, and agent services; the privacy boundary belongs in the extension/local layer.

5. Detection & Perception Design

Use a multi-signal decision rather than a single regex or model.

```
Entity candidate = { value/text, bounding_box, source: DOM | OCR | BOTH, category, confidence, reasons,
element_id_if_any } Sensitivity score = f( pattern evidence, DOM input type, label/nearby text, page context,
visual context, model confidence )
```

Initial categories

- Email, phone, name, postal address.
- Password, OTP, authentication token/session-like values.
- Payment/card/bank-like information for the synthetic benchmark.
- Government/identity-like identifiers using synthetic examples.
- User-defined custom sensitive patterns.
- Context-sensitive identifiers where the same-looking string can be safe or sensitive depending on its label and page context.

Decision example

Observed value	Evidence	Action
user@example.com	Email pattern + label 'Email' + email input	Protect
********	Password input type	Protect
₹2,499	Currency + label 'Price' + product context	Allow
827364	Order ID label + order page	Medium / configurable
827364	Product ID label	Usually allow

Key idea: The system should explain why it protected or allowed an item. Explainability is useful for debugging, judge confidence, and false-positive analysis.

6. Semantic Sanitization & Local Vault

Do not replace every secret with an unreadable black box. Preserve the semantic role.

```
Original page: Name: Raj Kumar Email: raj@example.com Phone: 9876543210 Sanitized agent context: Name:
USER_NAME_1 Email: USER_EMAIL_1 Phone: USER_PHONE_1 Local-only mapping: USER_NAME_1 -> actual value USER_EMAIL_1
-> actual value USER_PHONE_1 -> actual value
```

Alias requirements

- Stable during a task/session so the agent can reason consistently.
- Opaque: alias must not encode the real value.
- Typed: USER_EMAIL_1 is preferable to VALUE_7 because it preserves semantic type.
- Non-reversible by the remote agent.
- Resolved only inside the extension at the moment a browser action requires the value.
- Never include the alias→real-value mapping in prompts, logs, telemetry, or benchmark exports.

Action contract

```
Agent output should be structured, not arbitrary JavaScript: { "action": "TYPE", "target": "email_input",
"value": "USER_EMAIL_1" } Extension: 1. Validate action schema. 2. Verify target. 3. Resolve alias locally. 4.
Inject actual value into the target. 5. Record only metadata such as 'alias resolved', not the secret.
```

7. Privacy Firewall & Leakage Sentinel

Firewall flow

```
Agent context / tool call ↓ Normalize + inspect ↓ Find protected values and forbidden raw strings ↓
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ raw sensitive ■ safe/sanitized ■ ■ found ■ ■ ↓ ↓ BLOCK/REPLACE
ALLOW ↓ Sanitized request ↓ Remote agent
```

Leakage test strategy

- Generate a controlled webpage containing unique synthetic canary secrets.
- Register each canary with the local test harness.
- Run the browser-agent task.
- Capture/log the exact sanitized context and test-visible outgoing payloads.
- Search for exact canary strings and robust variants (case/encoding where relevant).
- Compute leakage rate and identify the component responsible if a canary appears.

Core metric

Leakage Rate = number of protected secret values observed outside the local boundary / number of protected secret values encountered.

Security caveat

For the SIH prototype, clearly define the threat model and what the extension can and cannot observe. Use an external controlled network test harness where necessary. Do not claim absolute security merely because the UI says 'protected'. The project wins credibility by stating its boundary and measuring it.

8. Browser-Agent Design

Use constrained structured actions.

Action	Example	Validation
CLICK	CLICK(button#submit)	Element exists + visible + allowed
TYPE	TYPE(input#email, USER_EMAIL_1)	Alias or safe text only
SELECT	SELECT(country, India)	Option exists
SCROLL	SCROLL(500)	Bounds check
NAVIGATE	NAVIGATE(approved URL)	Allowlist/policy

Agent context should contain

- Task objective.
- Sanitized page structure.
- Sanitized visible text.
- Semantic aliases where necessary.
- Available structured actions.
- Policy constraints and privacy mode.

Agent context should NOT contain

- Raw password values.
- Raw email/phone/address values when protected.
- Alias→real-value mappings.
- Unfiltered screenshots containing protected values.
- Arbitrary secrets embedded in debugging logs.

Recommended first agent

Start with a deterministic action planner or a simple LLM that emits JSON actions. Only after the action bridge is reliable should you add more autonomous browser behavior. This reduces debugging complexity and makes the demo reproducible.

9. Existing Work vs Your Novelty

Existing direction	What it already demonstrates	Your response
WebPII / WebRedact	Visual PII localization/redaction benchmark and detector	Use as conceptual/benchmark baseline; focus on agent utility after s
Browser-agent research	Agents can execute tasks on websites	Add a privacy boundary and prove raw PII does not reach the agent.
Simple extension PII blockers	Detect and blur sensitive fields	Semantic aliases + local vault + agent action bridge.
OCR-based masking	Find text and cover it	Fuse DOM + OCR + visual context and preserve task semantics.

What to claim

“Existing systems address pieces of the problem. PrivAgent integrates local multimodal perception, semantic privacy transformation, local reversible identity mapping, constrained browser actions, and measurable network-leakage testing into one end-to-end agent workflow.”

What NOT to claim

- Do not claim that nobody has built visual PII detection.
- Do not claim that nobody has built browser agents.
- Do not claim absolute security without a defined threat model and tests.
- Do not claim benchmark superiority until your measurements are complete.

10. SIH-Grade Benchmark & Metrics

Build a synthetic benchmark called PrivAgent-Bench.

Page/task families

- Registration/profile forms.
- Shopping/checkout-like forms.
- Travel/booking forms.
- College/administrative forms.
- Customer-support forms.
- Settings/account-management pages.
- Banking-like and medical-like pages using synthetic data only.
- Dynamic pages with popups, lazy-loaded fields, and visual-only content.

Difficulty levels

Level	Characteristics
Easy	PII is plainly labeled in standard DOM inputs.
Medium	Unusual layout, dynamic DOM, partially filled fields, ambiguous labels.
Hard	Text in images/canvas, visual-only elements, mixed sensitive/non-sensitive values, changing layouts.

Metrics

Metric	Meaning
PII recall	Fraction of sensitive items correctly detected.
False-positive rate	Fraction of safe items unnecessarily protected.
Leakage rate	Protected values observed outside the local boundary.
Task success rate	Tasks completed correctly after sanitization.
Local inference latency	Time for local perception/protection.
CPU/GPU/RAM	Resource cost on target hardware.
Network bytes	Context/data transmitted to agent service.
Protection overhead	Added latency/resource use compared with unprotected baseline.

11. Experimental Comparison

Always compare at least three systems.

Method	Privacy	Task utility	Purpose
A. No protection	Low	High baseline	Upper bound on task success; demonstrates risk
B. Full redaction	High	Often lower	Shows utility loss from destroying context.
C. PrivAgent	High target	High target	Tests semantic sanitization and local resolution.

The winning graph

```
Task Success ↑ ■ ● PrivAgent ■ ■ ● Partial/naive ■ ■ ■ ● Full redaction ■■■■■■■■■■■■■■■■■■■■■→ Privacy protection
```

The graph above is a target visualization, not a result. Replace it with measured data. The strongest result is a system that moves toward the top-right: high protection with minimal loss of task success.

Ablation experiments

- DOM-only vs vision-only vs fused DOM+vision.
- Black-box redaction vs semantic aliases.
- No firewall vs firewall.
- CPU vs WebGPU where hardware supports it.
- Context-aware sensitivity vs simple pattern matching.

12. Implementation Repository Structure

```
privagent/ ■■■ extension/ ■ ■■■ manifest.json ■ ■■■ src/ ■ ■■■ background/ ■ ■■■ content/ ■ ■■■ sidepanel/
■ ■■■ perception/ ■ ■ ■■■ dom/ ■ ■ ■■■ ocr/ ■ ■ ■■■ vision/ ■ ■■■ sensitivity/ ■ ■■■ sanitizer/ ■ ■■■
vault/ ■ ■■■ firewall/ ■ ■■■ agent/ ■ ■■■ actions/ ■ ■■■ telemetry/ ■■■ models/ ■ ■■■ onnx/ ■ ■■■
configs/ ■■■ backend/ ■ ■■■ fastapi/ ■■■ benchmark/ ■ ■■■ pages/ ■ ■■■ tasks/ ■ ■■■ canaries/ ■ ■■■
leakage/ ■ ■■■ reports/ ■■■ tests/ ■ ■■■ unit/ ■ ■■■ integration/ ■ ■■■ e2e/ ■■■ docs/ ■■■
architecture.md ■■■ threat-model.md ■■■ benchmark.md ■■■ demo-script.md
```

Engineering rule

Keep perception, sanitization, vault, firewall, and action execution as separate modules with explicit interfaces. This lets another AI or developer implement and test each component independently without mixing privacy-critical logic into UI code.

13. Interfaces an AI Developer Can Implement Directly

Core TypeScript-style data contracts

```
type SensitiveEntity = { id: string; category: "EMAIL" | "PHONE" | "NAME" | "ADDRESS" | "PASSWORD" | "OTP" | "PAYMENT" | "ID" | "CUSTOM"; source: "DOM" | "OCR" | "VISION" | "FUSED"; text?: string; // local only for protected entities bbox?: [number, number, number, number]; confidence: number; reasons: string[]; elementId?: string; }; type AliasRecord = { alias: string; // e.g. USER_EMAIL_1 category: string; sessionId: string; createdAt: number; // actualValue MUST remain local }; type AgentAction = | { action: "CLICK"; target: string } | { action: "TYPE"; target: string; value: string } | { action: "SELECT"; target: string; value: string } | { action: "SCROLL"; amount: number } | { action: "NAVIGATE"; url: string }; type PrivacyEvent = { type: "DETECTED" | "SANITIZED" | "BLOCKED" | "ALIAS_RESOLVED" | "TASK_RESULT"; entityCategory?: string; alias?: string; timestamp: number; // never store the raw protected value };
```

Critical invariants

- Invariant 1: raw protected values never enter remote-agent payloads.
- Invariant 2: alias mappings never enter remote-agent payloads.
- Invariant 3: logs never store raw protected values.
- Invariant 4: action executor accepts only validated structured actions.
- Invariant 5: alias resolution happens only in the local action bridge.
- Invariant 6: benchmark secrets are synthetic and uniquely identifiable.

14. Build Roadmap — 8 Weeks

Week	Build	Definition of done
1	MV3 extension + DOM collector + side panel	Extension reads a test page and displays structured page elements.
2	Rule-based PII detector	Email/phone/password/name/address detection works on benchmark pages.
3	Screenshot + OCR	Visual-only text can be detected with coordinates.
4	ONNX/WebGPU local inference + fusion	Core perception runs locally with measured latency.
5	Semantic aliases + local vault	Agent context contains aliases; real values stay local.
6	Agent + structured action bridge	Agent completes simple tasks using aliases.
7	Privacy firewall + leakage sentinel	Known synthetic secrets are detected and leakage is measurable.
8	Benchmark + dashboard + demo hardening	A repeatable 5-minute demo and measured comparison are ready.

Team split for 4–5 students

- Person 1 — Extension/platform: Manifest V3, DOM, side panel, action bridge.
- Person 2 — Local AI: OCR, ONNX/WebGPU, model optimization.
- Person 3 — Privacy/security: sensitivity engine, sanitizer, vault, firewall, leakage testing.
- Person 4 — Agent/backend: FastAPI, LLM orchestration, structured tools, benchmark runner.
- Person 5 (optional) — UI/benchmark/research: dashboard, datasets, metrics, documentation, presentation.

15. Winning 5-Minute Demonstration

Time	Demo moment	Judge takeaway
0:00–0:30	Show the problem: an AI agent normally sees sensitive page data.	Problem is clear.
0:30–1:00	Open a realistic synthetic form with name/email/phone/address/password.	Realistic use case.
1:00–1:30	Enable PrivAgent; show local detection and privacy heatmap.	Perception works.
1:30–2:00	Show aliases: USER_EMAIL_1 etc.; emphasize mapping stays local.	Novel privacy mechanism.
2:00–3:00	Give the agent a task; it completes it using sanitized context.	Utility is preserved.
3:00–4:00	Show leakage sentinel/network test with zero raw canary values observed.	Privacy is measured, not claimed.
4:00–4:30	Compare full redaction vs semantic sanitization.	Your approach solves the privacy–utility problem.
4:30–5:00	Show benchmark metrics and architecture; deliver closing statement.	Research + engineering maturity.

Recommended closing line

“An AI agent should be able to act on your behalf without needing to know who you are.”

16. Final Acceptance Checklist

- The extension runs on Chromium/Chrome with Manifest V3.
- DOM-based sensitive-data detection works.
- Visual/OCR-based detection works for selected visual-only cases.
- Detection combines context, not just regex.
- Protected values become semantic aliases before agent transmission.
- Alias→real-value mappings stay local.
- The remote agent receives only sanitized context.
- The agent can execute structured browser actions.
- The action bridge resolves aliases locally when required.
- The privacy firewall checks outgoing agent context.
- A controlled leakage test uses synthetic canary secrets.
- The benchmark reports PII recall, false positives, leakage, task success, latency and resource usage.
- Ablation experiments show why each major component matters.
- The threat model clearly states what is protected and what is outside scope.
- The demo never uses real personal data.
- The project has a reproducible setup guide so another developer/AI can build it.

Definition of “winning-ready”

The project is winning-ready when you can demonstrate the same task three ways: (1) unprotected agent — high utility but privacy exposure, (2) full redaction — high privacy but degraded utility, and (3) PrivAgent — high privacy with substantially preserved task utility. The measured comparison, not the number of features, is the centerpiece.

17. Copy-Paste Implementation Brief for Another AI

Use the following as the implementation handoff. It is intentionally explicit so a coding AI can turn the plan into a repository without inventing the product direction.

```
PROJECT: PrivAgent — SIH 2026 GOAL: Build a Chrome/Chromium Manifest V3 browser extension that enables an AI browser agent to complete web tasks while sensitive user data remains local. NON-NEGOTIABLE PRIVACY RULES: 1. Detect sensitive data locally whenever feasible. 2. Never send raw protected values to the remote agent. 3. Never send alias-to-real-value mappings to the remote agent. 4. Never log raw protected values. 5. Resolve aliases only in the local action bridge. 6. Use synthetic data for all development/demo benchmarks. CORE FEATURES: - DOM/accessibility perception - Visual screenshot perception - OCR - Context-aware sensitivity classification - Semantic aliases - Local vault - Privacy firewall - Structured browser actions - Leakage sentinel - Privacy/utility dashboard - Benchmark runner STACK: TypeScript + React + Vite + Manifest V3 ONNX Runtime Web + WebGPU/CPU fallback OCR implementation compatible with browser execution Python + FastAPI for agent/backend services Playwright for end-to-end testing IndexedDB/extension storage for local metadata IMPLEMENTATION ORDER: 1. Extension skeleton 2. DOM collector 3. Rule-based detector 4. OCR/visual collector 5. Local model integration 6. Fusion/sensitivity engine 7. Semantic sanitizer 8. Local alias vault 9. Agent gateway 10. Structured action bridge 11. Privacy firewall 12. Leakage sentinel 13. Benchmark 14. Dashboard 15. SIH demo hardening SUCCESS CRITERIA: - The agent completes realistic benchmark tasks after sanitization. - Raw synthetic PII does not appear in the agent context. - Leakage rate is measured. - Task success is measured. - Local latency and resource usage are measured. - Comparison against no protection and full redaction is included. - All major components have tests. DO NOT: - Build only a black-box PII extension. - Claim novelty for PII detection alone. - Send raw PII to a cloud backend for “processing”. - Train a giant model before the end-to-end prototype works. - Add uncontrolled arbitrary JavaScript execution to the agent.
```

End of implementation blueprint

Use this document as the master product/engineering specification. Implementation details such as exact model weights, browser API versions, and LLM provider can be selected during development as long as the privacy boundary, semantic sanitization, local alias resolution, leakage testing, and measured task utility remain intact.